

Solutions to Kalman Filter Assignment

Advanced Engineering Analysis

June 29, 2026

1 Analytical Derivation

[1pt]

Consider the filter model:

$$\begin{aligned}x_k^- &= A_{k-1} x_{k-1}^+ + G_{k-1} w_{k-1}, \\y_k^- &= H_k \\&\quad, x_k^- + z_k,\end{aligned}$$

where $x_{k-1}^+ \sim \mathcal{N}(m_{k-1}, P_{k-1})$, $w_{k-1} \sim \mathcal{N}(0, \Sigma_p)$, and $z_k \sim \mathcal{N}(0, \Sigma_m)$ are mutually independent.

1.1 1. Distribution of the Predicted State x_k^-

We wish to show that $x_k^- \sim \mathcal{N}(m_k^-, P_k^-)$, where:

$$m_k^- = A_{k-1} m_{k-1}, \quad P_k^- = A_{k-1} P_{k-1} A_{k-1}^T + G_{k-1} \Sigma_p G_{k-1}^T.$$

Proof: Since x_k^- is a linear combination of two independent Gaussian random variables x_{k-1}^+ and w_{k-1} , it is itself Gaussian. We find its mean and covariance using the linearity of expectation and properties of covariance matrices:

$$\begin{aligned}m_k^- &= \mathbb{E}[x_k^-] = \mathbb{E}[A_{k-1} x_{k-1}^+ + G_{k-1} w_{k-1}] \\&= A_{k-1} \mathbb{E}[x_{k-1}^+] + G_{k-1} \mathbb{E}[w_{k-1}] \\&= A_{k-1} m_{k-1} + G_{k-1} (0) = A_{k-1} m_{k-1}.\end{aligned}$$

For the covariance matrix P_k^- :

$$\begin{aligned}P_k^- &= \text{Var}(x_k^-) = \text{Var}(A_{k-1} x_{k-1}^+ + G_{k-1} w_{k-1}) \\&= A_{k-1} \text{Var}(x_{k-1}^+) A_{k-1}^T + G_{k-1} \text{Var}(w_{k-1}) G_{k-1}^T \\&= A_{k-1} P_{k-1} A_{k-1}^T + G_{k-1} \Sigma_p G_{k-1}^T.\end{aligned}$$

Thus, $x_k^- \sim \mathcal{N}(m_k^-, P_k^-)$.

1.2 2. Distribution of the Predicted Measurement y_k^-

We wish to show that $y_k^- \sim \mathcal{N}(H_k m_k^-, H_k P_k^- H_k^T + \Sigma_m)$.

Proof: Since y_k^- is a linear function of the Gaussian random variable x_k^- and an independent Gaussian noise z_k , y_k^- is Gaussian.

$$\mathbb{E}[y_k^-] = \mathbb{E}[H_k x_k^- + z_k] = H_k \mathbb{E}[x_k^-] + \mathbb{E}[z_k] = H_k m_k^-.$$

Using independence between x_k^- and z_k :

$$\begin{aligned}\text{Var}(y_k^-) &= \text{Var}(H_k x_k^- + z_k) = H_k \text{Var}(x_k^-) H_k^T + \text{Var}(z_k) \\&= H_k P_k^- H_k^T + \Sigma_m.\end{aligned}$$

Thus, $y_k^- \sim \mathcal{N}(H_k m_k^-, H_k P_k^- H_k^T + \Sigma_m)$.

1.3 3. Joint Distribution of x_k^- and y_k^-

We wish to show that:

$$\begin{bmatrix} x_k^- \\ y_k^- \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m_k^- \\ H_k m_k^- \end{bmatrix}, \begin{bmatrix} P_k^- & P_k^- H_k^T \\ H_k P_k^- & H_k P_k^- H_k^T + \Sigma_m \end{bmatrix} \right).$$

Proof: The mean vector follows directly from parts 1 and 2. The cross-covariance terms are derived as follows:

$$\begin{aligned} \text{Cov}(x_k^-, y_k^-) &= \mathbb{E}[(x_k^- - m_k^-)(y_k^- - H_k m_k^-)^T] \\ &= \mathbb{E}[(x_k^- - m_k^-)(H_k x_k^- + z_k - H_k m_k^-)^T] \\ &= \mathbb{E}[(x_k^- - m_k^-)(x_k^- - m_k^-)^T H_k^T + (x_k^- - m_k^-)z_k^T] \\ &= \text{Var}(x_k^-) H_k^T + 0 = P_k^- H_k^T. \end{aligned}$$

By symmetry, $\text{Cov}(y_k^-, x_k^-) = H_k P_k^-$. Assembling these blocks yields the joint covariance matrix.

1.4 4. Conditional Distribution (The Kalman Update Step)

We wish to show that $x_k^+ \triangleq (x_k^- | y_k^- = y_k^{\text{obs}}) \sim \mathcal{N}(m_k, P_k)$, where:

$$\begin{aligned} K_k &\triangleq P_k^- H_k^T (H_k P_k^- H_k^T + \Sigma_m)^{-1}, \\ m_k &= m_k^- + K_k (y_k^{\text{obs}} - H_k m_k^-), \\ P_k &= (I - K_k H_k) P_k^-. \end{aligned}$$

Proof: Using the standard rule for conditional distributions of multivariate Gaussians, if a joint vector is partitioned with mean blocks μ_x, μ_y and covariance blocks $\Sigma_{xx}, \Sigma_{xy}, \Sigma_{yy}$, then the conditional mean and covariance are:

$$\mu_{X|Y} = \mu_x + \Sigma_{xy} \Sigma_{yy}^{-1} (y - \mu_y), \quad \Sigma_{X|Y} = \Sigma_{xx} - \Sigma_{xy} \Sigma_{yy}^{-1} \Sigma_{yx}.$$

Substituting $\mu_x = m_k^-$, $\mu_y = H_k m_k^-$, $\Sigma_{xx} = P_k^-$, $\Sigma_{xy} = P_k^- H_k^T$, and $\Sigma_{yy} = H_k P_k^- H_k^T + \Sigma_m$, and setting $K_k = \Sigma_{xy} \Sigma_{yy}^{-1}$, we get the standard Kalman update equations.

1.5 5. Expected Value and Variance

The requested values are precisely the conditional mean and covariance from the update step:

$$\mathbb{E}[x_k^- | y_k^- = y_k^{\text{obs}}] = m_k, \quad \text{Var}(x_k^- | y_k^- = y_k^{\text{obs}}) = P_k.$$

2 1-D Example [1pt]

2.1 1. Predictive Mean and Variance

$$m_k^- = a m_{k-1}, \quad P_k^- = a^2 P_{k-1} + q.$$

2.2 2. Updated Mean and Variance

Defining the innovation variance $S_k = h^2 P_k^- + r$:

$$m_k = m_k^- + \frac{P_k^- h}{S_k} (y_k^{\text{obs}} - h m_k^-), \quad P_k = \left(1 - \frac{P_k^- h^2}{S_k}\right) P_k^-.$$

2.3 3. Predictive Measurement Distribution

$$p(y_k^- | Y_{k-1}) = \mathcal{N}(hm_k^-, h^2 P_k^- + r).$$

2.4 4. Posterior-Predictive Measurement Distribution

$$p(y_k^- | Y_k) = \mathcal{N}(hm_k, h^2 P_k + r).$$

2.5 5. Numerical Example Script

```
import numpy as np

# Simulation parameters
a, q, h, r = 1.0, 0.5, 1.0, 0.2
m0, P0 = 0.0, 1.0
y_obs = 1.5

m1_minus = a * m0
P1_minus = (a**2) * P0 + q
S1 = (h**2) * P1_minus + r
K1 = (P1_minus * h) / S1
m1 = m1_minus + K1 * (y_obs - h * m1_minus)
P1 = (1 - K1 * h) * P1_minus

print(f"Prior Prediction: Mean = {m1_minus:.2f}, Var = {P1_minus:.2f}")
print(f"Posterior Filtered: Mean = {m1:.2f}, Var = {P1:.2f}")
```

3 2D-Position Estimation

[1pt]

3.1 Part A: System Matrix Derivations

From kinematic motion rules with a constant velocity model where velocity changes are modeled as a random acceleration process noise $w_k = [a_x, a_y]^T$:

$$p_x(k) = p_x(k-1) + v_x(k-1)\Delta t + \frac{1}{2}a_x\Delta t^2$$

$$v_x(k) = v_x(k-1) + a_x\Delta t$$

Arranging these in matrix form gives the transition matrix A , noise injection matrix G , and position-only measurement matrix H as requested.

3.2 Part B: Implementation Function

```
class KalmanFilter2D:
    def __init__(self, dt, sigma_p, sigma_m):
        self.dt = dt
        self.A = np.array([[1, 0, dt, 0], [0, 1, 0, dt], [0, 0, 1, 0], [0, 0, 0, 1]])
        self.H = np.array([[1, 0, 0, 0], [0, 1, 0, 0]])
        self.G = np.array([[0.5*dt**2, 0], [0, 0.5*dt**2], [dt, 0], [0, dt]])
        self.Q = self.G @ (sigma_p**2 * np.eye(2)) @ self.G.T
        self.R = sigma_m**2 * np.eye(2)
        self.x = np.zeros((4, 1))
        self.P = np.eye(4) * 10.0
```

```

def predict(self):
    self.x = self.A @ self.x
    self.P = self.A @ self.P @ self.A.T + self.Q

def update(self, y_meas):
    y = np.array(y_meas).reshape(2, 1)
    S = self.H @ self.P @ self.H.T + self.R
    K = self.P @ self.H.T @ np.linalg.inv(S)
    self.x = self.x + K @ (y - self.H @ self.x)
    self.P = (np.eye(4) - K @ self.H) @ self.P

```